

4

4. Strutture dati "complesse"

Tipi di dati - *cosa*

un **tipo di variabile** è un attributo che specifica l'insieme di valori che la variabile può assumere e le relative operazioni.

Esempio: classe in `java`

Una classe in `java` è costituita da:

- dati - oggetti
- operazioni - metodi

in passato venivano anche detti tipi di dato *astratti*

Esempio: tipo dizionario



Collezioni di elementi ciascuno dei quali è caratterizzato da una chiave

Chiavi: appartengono ad un dominio totalmente ordinato

operazioni tipiche:

- inserimento
- ricerca
- cancellazione

Strutture dati - *come*

Specifica organizzazione delle informazioni che permette di realizzare e implementare un determinato tipo di dati.

stesso tipo $\neq$ stessa struttura dati

esempio: dizionario

- array ordinato in base alle chiave
 - ricerca - $\Theta(\log n)$
 - inserimento - $\Theta(n)$
- array non ordinato
 - ricerca - $\Theta(n)$
 - inserimento - $O(1)$

Quale scegliere? dipende dalle specifiche.

Strutture indicizzate - array

- posizione contigua in memoria
- accesso mediante indice
- tempo di accesso costante indipendente dalla posizione del dato

limite: capacità fissata

Classificati come: *strutture statiche*

Strutture collegate

- non è necessario allocare la struttura in una porzione contigua in memoria

- elementi collegati fra loro
- passaggio di un elemento ad altri tramite collegamenti
- varie tipologie di collegamento (liste, alberi, ...)

Classificate come: *strutture dinamiche*

Liste concatenate

lista lineare

Insieme ordinato di nodi collegati linearmente uno dopo l'altro

ogni nodo contiene:

- un dato della collezione (compresa una chiave)
- l'informazione per accedere al nodo successivo

Come inserire un elemento in una lista ordinata?

<i>prossimo</i>
<i>chiave</i>
<i>altri campi</i>

- accesso tramite nodi riferimenti (puntatori)

`x` = riferimento al nodo

`x.chiave` = campo chiave

`x.pros` = riferimento a nodo successivo

`null` → riferimento nullo

Ricerca elemento in base alla chiave

```
FUNZIONE elemento(Lista L, intero i) -> Nodo
  p <- L
  WHILE p != null AND i > 0 DO
    p <- p.pros
    i <- i - 1
  RETURN p
```

Ricerca elemento in base alla chiave

```
FUNZIONE trova(Lista L, tipoChiave k) -> Nodo
  p <- L
  WHILE p != null AND p.chiave != k DO
    p <- p.pros
  RETURN p
```

Ricerca elemento in base alla chiave in una lista ordinata

```
FUNZIONE trova(ListaOrdinata L, tipoChiave k) -> Node
  p <- L
  WHILE p != null AND p.chiave < k DO
    p <- p.pros
  IF p = null OR p.chiave > k THEN
    RETURN null
  ELSE RETURN p
```

Inserimento in una lista ordinata

```

FUNZIONE inserisci(ListaOrdinata L, elemento d) -> ListaOrdinata
  k <- d.chiave
  p <- L
  prec <- null // uso due puntatori, uno che "segue"
  WHILE p != null AND p.chiave < k DO
    prec <- p
    p <- p.pros
  r <- riferimento a nuovo nodo
  r.chiave <- k
  r.altricampi <- d.altricampi
  r.pros <- p
  if prec = null THEN
    L <- r
  ELSE prec.pros <- r // attenzione
  RETURN L

```



Lezione 13 - @October 22, 2025

Cancellazione

- `prec.pros <- p.pros` - *rilasciare la memoria*
- caso particolare: cancellazione all'inizio

Esercizio

provare ad implementare gli algoritmi di ordinamento visti fino ad ora ed usare le `linked list`, le prestazioni sono simili?

Strutture astratte

Pila - `stack`

Struttura LIFO

Operazioni

- `isEmpty()` → boolean
- `push(elemento)`
- `pop()` → elemento
- `top()` → elemento

Implementazione tramite array

```

FUNZIONE isEmpty() -> boolean
  IF top = -1 THEN RETURN true
  ELSE RETURN false

```

```

PROCEDURA push(elemento x) // se capacità massima raggiunta, errore!!
  top <- top + 1
  dati[top] <- x

```

```

FUNZIONE top() -> elemento // errore se pila vuota!
  RETURN dati[top]

```

```
FUNZIONE pop() -> elemento // errore se pila vuota!  
  x <- dati[top]  
  top <- top - 1  
  RETURN x
```

Operazioni in tempo costante $O(1)$

Implementazione tramite lista concatenata

→ *voglio mantenere il tempo delle operazioni costante*

```
FUNZIONE isEmpty() -> boolean  
  IF top = null THEN  
    RETURN TRUE  
  ELSE  
    RETURN FALSE
```

```
PROCEDURA push(elemento x)  
  r <- riferimento ad un nuovo nodo  
  r.dati <- x  
  r.pros <- top  
  top <- r
```

```
FUNZIONE top() -> elemento // se pila vuota errore.  
  RETURN top.dato
```

```
FUNZIONE pop() -> elemento  
  x <- top.dato  
  top <- top.pros  
  // rilascio memoria  
  RETURN x
```

Tempo operazioni $O(1)$

Coda - **queue**

Struttura FIFO

Operazioni

- isEmpty() → boolean

```
FUNZIONE isEmpty() -> boolean  
  IF primo = null THEN  
    return TRUE  
  ELSE RETURN false
```

- enqueue(elemento)

```
PROCEDURA enqueue(elemento x)  
  r <- riferimento a un nuovo nodo  
  r.dato <- x  
  IF primo = null  
    primo <- r  
    ultimo <- r  
  ELSE
```

```
ultimo.pros <- r
ultimo <- r
```

- dequeue() → elemento

```
FUNZIONE dequeue() -> elemento
  x <- primo.dato
  primo <- primo.pros
  IF primo = null THEN
    ultimo <- null
  RETURN x
```

- first() → elemento

```
FUNZIONE first() -> elemento
  RETURN primo.dato
```

Tempo costante $O(1)$